

14. Ejercicio de CoT: BookingCalendar

Jonathan Ricardo Serna Galina · 20074902

1. Descripción de CoT y del proyecto

CoT (Chain of Thought) es una técnica de prompting que utiliza pasos intermedios para abordar un problema. Una forma de aplicarla es proporcionar ejemplos resueltos antes de plantear una nueva tarea. En este ejercicio se muestra una explicación organizada de la solución.

El proyecto consiste en crear la clase BookingCalendar para reservar salas de reuniones durante un día. Debe evitar reservas que se encimen en la misma sala, permitir cancelaciones y mostrar los horarios libres en una tabla de 24 horas.

2. Charla con la IA y desarrollo

Diálogo ilustrativo para demostrar el método; no es una transcripción de la conversación original.

Estudiante: Necesito una clase BookingCalendar. Ayúdame a organizar la solución: primero identifica los datos, después explica las reglas de reserva y cancelación, y finalmente genera el código y una prueba.

IA: Cada reserva necesita sala, hora de inicio y hora de fin. La clase tendrá tres métodos: book para reservar, cancel para cancelar y get_available_slots para consultar los espacios libres.

Estudiante: ¿Cómo se distingue una reserva que se encima de una que empieza cuando termina otra?

IA: En la misma sala, 09:00-11:00 y 10:00-12:00 se enciman. En cambio, 09:00-11:00 y 11:00-12:00 son compatibles. Las reservas de salas diferentes se revisan por separado.

Estudiante: También quiero rechazar horarios inválidos y ver la disponibilidad después de cancelar.

IA: El inicio debe ser menor que el fin. La cancelación debe coincidir con la sala y el horario completos. La disponibilidad se obtiene de los espacios que quedan entre las reservas ordenadas, desde 00:00 hasta 24:00.

Estudiante: Genera la clase siguiendo PEP 8 y una demostración con una reserva válida, un conflicto y una cancelación.

Resultado: La implementación y su demostración se incluyen a continuación. La salida real se presenta en la sección 4.

Referencia: Wei et al. (2022). [Chain-of-Thought Prompting Elicits Reasoning in Large Language Models](#).

3. Diagrama y código como resultado

BookingCalendar

_bookings: diccionario de reservas por sala

```
+ book(start, end, room) -> bool
+ cancel(start, end, room) -> bool
+ get_available_slots(room=None) -> list
- _validate(start, end, room) -> tuple
- _to_minutes(value) -> int
- _format_time(minutes) -> str
```

La clase guarda una lista de horarios por sala. El signo + indica métodos públicos y el signo - detalles internos. main.py usa una instancia para demostrar las operaciones.

main.py: demostración

```
"""Demostración reproducible de BookingCalendar."""

from booking_calendar import BookingCalendar

def main():
    calendar = BookingCalendar()
    operations = [
        (8, 10, "Sala 1"),
        (10, 12, "Sala 1"),
        (9, 11, "Sala 1"),
        (14, 16, "Sala 1"),
        (16, 17, "Sala 2"),
        (14, 14, "Sala 1"),
        ("18:30", "19:15", "Sala 2"),
    ]
    for start, end, room in operations:
        result = calendar.book(start, end, room)
        print(f"Reservar {room} ({start}, {end}): {result}")
    print("Cancelar inexistente:", calendar.cancel(9, 11, "Sala 3"))
    print("Cancelar Sala 1 (14, 16):", calendar.cancel(14, 16, "Sala 1"))
    print("\nDisponibilidad después de cancelar:")
    calendar.get_available_slots()

if __name__ == "__main__":
    main()
```

booking_calendar.py

```
1  """Reservas de salas para un día, sin dependencias externas."""
2
3  import re
4
5
6  class BookingCalendar:
7      """Administra intervalos [inicio, fin) independientes por sala.
8
9      Acepta horas enteras o cadenas HH:MM. 24:00 solo puede ser fin.
10     Las salas distinguen mayúsculas y eliminan espacios exteriores.
11     """
12
13     def __init__(self):
14         self._bookings = {}
15
16     @staticmethod
17     def _to_minutes(value):
18         """Convierte una hora válida a minutos desde medianoche."""
19         if type(value) is int and 0 <= value <= 24:
20             return value * 60
21         if isinstance(value, str) and re.fullmatch(r"\d{2}:\d{2}", value):
22             hour, minute = map(int, value.split(":"))
23             if 0 <= hour < 24 and 0 <= minute < 60:
24                 return hour * 60 + minute
25             if hour == 24 and minute == 0:
26                 return 1440
27             raise ValueError("Usa horas enteras de 0 a 24 o el formato HH:MM.")
28
29     @staticmethod
30     def _format_time(minutes):
31         """Devuelve una hora de 24 horas con minutos."""
32         return f"{minutes // 60:02d}:{minutes % 60:02d}"
33
34     def _validate(self, start, end, room):
35         """Valida el intervalo y el nombre antes de modificar datos."""
36         if not isinstance(room, str) or not room.strip():
37             raise ValueError("El nombre de la sala no puede estar vacío.")
38         start = self._to_minutes(start)
39         end = self._to_minutes(end)
40         if not 0 <= start < end <= 1440:
41             raise ValueError("El inicio debe ser anterior al fin del día.")
42         return start, end, room.strip()
43
44     def book(self, start, end, room):
45         """Reserva si los datos son válidos y no existe un traslape."""
46         try:
47             start, end, room = self._validate(start, end, room)
48         except ValueError:
49             return False
50         bookings = self._bookings.get(room, [])
51         if any(start < old_end and old_start < end
52               for old_start, old_end in bookings):
53             return False
54         self._bookings.setdefault(room, []).append((start, end))
```

booking_calendar.py (continuación 1)

```
55         self._bookings[room].sort()
56         return True
57
58     def cancel(self, start, end, room):
59         """Cancela únicamente una coincidencia exacta de sala y horario."""
60         try:
61             start, end, room = self._validate(start, end, room)
62         except ValueError:
63             return False
64         bookings = self._bookings.get(room, [])
65         if (start, end) not in bookings:
66             return False
67         bookings.remove((start, end))
68         return True
69
70     def get_available_slots(self, room=None):
71         """Imprime una tabla y devuelve filas con sala, inicio y fin.
72
73         Sin argumento consulta salas registradas. Una sala nueva que se
74         consulte explícitamente aparece libre durante todo el día.
75         """
76         if room is not None:
77             if not isinstance(room, str) or not room.strip():
78                 raise ValueError("Indica un nombre de sala válido.")
79             rooms = [room.strip()]
80         else:
81             rooms = sorted(self._bookings)
82         rows = []
83         for name in rooms:
84             cursor = 0
85             for start, end in self._bookings.get(name, []):
86                 if cursor < start:
87                     rows.append((name, self._format_time(cursor),
88                                 self._format_time(start)))
89                     cursor = end
90                 if cursor < 1440:
91                     rows.append((name, self._format_time(cursor), "24:00"))
92         width = max([4] + [len(name) for name in rooms])
93         print(f"{'Sala':<{width}} | Inicio | Fin")
94         print("-" * (width + 17))
95         for name, start, end in rows:
96             print(f"{name:<{width}} | {start} | {end}")
97         if not rows:
98             print("Sin huecos libres o sin salas registradas.")
99         return rows
```

4. Ejecución

Desde la carpeta del ejercicio, se ejecuta:

```
python3 main.py
```

Salida real del programa:

```
Reservar Sala 1 (8, 10): True
Reservar Sala 1 (10, 12): True
Reservar Sala 1 (9, 11): False
Reservar Sala 1 (14, 16): True
Reservar Sala 2 (16, 17): True
Reservar Sala 1 (14, 14): False
Reservar Sala 2 (18:30, 19:15): True
Cancelar inexistente: False
Cancelar Sala 1 (14, 16): True

Disponibilidad después de cancelar:
Sala  | Inicio | Fin
-----
Sala 1 | 00:00 | 08:00
Sala 1 | 12:00 | 24:00
Sala 2 | 00:00 | 16:00
Sala 2 | 17:00 | 18:30
Sala 2 | 19:15 | 24:00
```

Las reservas de 08:00-10:00 y 10:00-12:00 se aceptan. La de 09:00-11:00 se rechaza por conflicto y la de 14:00-14:00 por duración nula. Al cancelar 14:00-16:00, Sala 1 queda libre desde las 12:00 hasta las 24:00.

Verificación adicional: el archivo del profesor y las 13 pruebas complementarias finalizaron correctamente.

Liga del código: [Abrir la carpeta del ejercicio en GitHub](#)